Distributed AI Inference on Mobile Mesh Networks

Pipeline Parallelism for Heterogeneous ~~Edge Devices on the Pars Network~~

Cyrus Pahlavi, Pars Teamresearch@pars.id

February 2026

Abstract

We present ParsInfer, a distributed inference framework that partitions large language models across heterogeneous mobile and edge devices connected via the Pars mesh network. ParsInfer enables community-scale AI inference without centralized GPU infrastructure by decomposing transformer models into pipeline stages optimally assigned to available devices based on their compute capacity, memory, and network bandwidth. We formulate the model partitioning problem as a constrained optimization over a weighted device graph and present an efficient dynamic programming solution with $O(L \cdot N^2)$ complexity for L layers and N devices. Our fault tolerance mechanism provides seamless recovery from device departure with median failover latency of 180ms. Evaluation on ParsLM models (1.3B–13B parameters) across realistic mesh topologies demonstrates that ParsInfer achieves 73% of single-GPU throughput using 8 consumer smartphones, with time-to-first-token latency of 420ms for the 7B model. We show that inference quality is bit-identical to single-device execution, as our partitioning operates at the layer boundary with no approximation.

1 Introduction

Large language models (LLMs) require substantial computational resources for inference, typically served from centralized GPU clusters [Pope et al., 2023]. For the Pars Network, this centralized dependency is problematic: it creates reliance on cloud providers who may restrict access due to sanctions or political pressure, and it requires funneling potentially sensitive community queries through centralized endpoints.

The Pars Network's mesh of community-operated devices—smartphones, tablets, laptops, and home servers—collectively possesses significant compute capacity. A typical diaspora household with a modern smartphone (6 GB RAM, Neural Engine), a laptop (16 GB RAM, discrete GPU), and a smart TV or gaming console provides enough aggregate compute for LLM inference at useful throughput, provided the model can be efficiently distributed across these devices.

1.1 Challenges

(1) **Device heterogeneity:** Available devices range from budget Android phones (2 GB RAM, Cortex-A55) to gaming PCs (32 GB RAM, RTX 4090). The partitioning must account for 100× differences in compute capability.

(2) **Network variability:** Devices connect via Wi-Fi (10–500 Mbps), Bluetooth (2 Mbps), or the Pars mesh network (1–50 Mbps). Link quality fluctuates over time.

(3) **Dynamic availability:** Devices join and leave the network unpredictably. A phone may be used for other tasks, a laptop may enter sleep mode, a user may leave Wi-Fi range.

(4) **Latency sensitivity:** Interactive AI applications (chat, search) require low time-to-first-token (TTFT) latency, constraining the pipeline depth and inter-device communication overhead.

(5) **Privacy:** Intermediate activations transmitted between devices could leak information about the input. The inference must preserve input confidentiality.

1.2 Contributions

(i) We formalize the *heterogeneous pipeline partitioning* problem and present a dynamic programming algorithm that finds optimal layer-to-device assignments in $O(L \cdot N^2)$ time.

(ii) We design a *speculative pipeline execution* strategy that overlaps communication with computation to hide inter-device latency.

(iii) We implement *hot-standby replication* for fault tolerance, maintaining redundant model shards on nearby devices with 180ms median failover.

(iv) We introduce *activation encryption* using lightweight stream ciphers to protect intermediate activations in transit, adding less than 3% overhead.

(v) We evaluate ParsInfer on realistic mesh topologies with consumer devices, demonstrating practical throughput for interactive LLM applications.

2 Related Work

2.1 Model Parallelism

Pipeline parallelism [Huang et al., 2019, Narayanan et al., 2019] distributes model layers across devices and processes micro-batches in a pipelined fashion. GPipe [Huang et al., 2019] introduced synchronous pipeline parallelism with gradient accumulation. PipeDream [Narayanan et al., 2019] uses asynchronous pipelines with weight stashing for training. These systems assume homogeneous GPU clusters with high-bandwidth interconnects.

2.2 Edge and Mobile Inference

Edge inference optimization includes model compression [Han et al., 2016], knowledge distillation [Hinton et al., 2015], and neural architecture search for efficient models [Cai et al., 2019]. Split computing [Kang et al., 2017] partitions DNNs between edge and cloud. DeepThings [Zhao et al., 2018] distributes CNN inference across IoT devices using fused-tile partitioning.

2.3 Collaborative Inference

Petals [Borzunov et al., 2023] enables collaborative inference of LLMs across volunteer-contributed GPUs. SWARM parallelism [Ryabinin et al., 2023] provides fault-tolerant pipeline parallelism for unreliable networks. Our work differs in targeting mobile/edge devices rather than GPUs, requiring fundamentally different partitioning strategies due to memory constraints and compute heterogeneity.

3 System Model

3.1 Device Graph

We model the available devices as a weighted graph $G = (V, E, w_v, w_e)$ where:

Definition 1 (Device Graph).

- $V = \{v_1, \ldots, v_N\}$ *is the set of available devices.*
- $E \subseteq V \times V$ *represents network connectivity.*
- $w_v : V \to \mathbb{R}_+^3$ *maps each device to its capabilities:* (c_i, m_i, s_i) *representing compute (FLOPS), available memory (bytes), and storage (bytes).*
- $w_e : E \to \mathbb{R}_+^2$ *maps each edge to* (b_{ij}, l_{ij}) *representing bandwidth (bytes/s) and latency (seconds).*

3.2 Model Representation

A transformer model with L layers is represented as a sequence of computational stages:

Definition 2 (Model Stage)**.** *Each layer $\ell \in \{1, \ldots, L\}$ is characterized by:*

- p_ℓ*: Parameter count (bytes when quantized).*
- f_ℓ*: FLOPs per token for forward pass.*
- a_ℓ*: Activation size (bytes) for a single token.*
- k_ℓ*: KV-cache size per token (bytes).*

For standard transformer architectures, these values are uniform across layers (except the embedding and output layers), but our formulation handles non-uniform architectures.

4 Optimal Model Partitioning

4.1 Problem Formulation

Definition 3 (Partitioning Problem)**.** *Given a model with L layers and a device graph G with N devices, find a partition $\pi : [L] \to [N]$ that minimizes the end-to-end inference latency:*

$$\min_{\pi} \max_{i \in [N]} \left(T_{comp}(i, \pi) + T_{comm}(i, \pi) \right) \tag{1}$$

subject to:

$$\sum_{\ell : \pi(\ell) = i} p_\ell + k_\ell \cdot S \leq m_i, \quad \forall i \in [N] \tag{2}$$

$$\pi(\ell) = \pi(\ell') \Rightarrow \pi(\ell'') = \pi(\ell), \quad \forall \ell \leq \ell'' \leq \ell' \tag{3}$$

where S is the maximum sequence length and constraint (3) *requires contiguous layer assignment (no layer splitting across devices).*

The compute time for device i processing its assigned layers is:

$$T_{\text{comp}}(i, \pi) = \frac{\sum_{\ell : \pi(\ell) = i} f_\ell}{c_i} \tag{4}$$

The communication time between consecutive pipeline stages is:

$$T_{\text{comm}}(i, \pi) = \frac{a_{\max(\ell : \pi(\ell) = i)}}{b_{\pi(i), \pi(i+1)}} + l_{\pi(i), \pi(i+1)} \tag{5}$$

4.2 Dynamic Programming Solution

The contiguity constraint (3) enables a dynamic programming solution:

Algorithm 1 Optimal Pipeline Partitioning

Require: Layers $1, \ldots, L$; devices $1, \ldots, N$ (sorted by pipeline position); device capabilities; network topology

Ensure: Optimal partition π^* minimizing pipeline latency

```
dp[ℓ][i] ← ∞ for all ℓ, i                          ▷ Min latency for layers 1..ℓ on devices 1..i
dp[0][0] ← 0
for i = 1 to N do
    for ℓ = 1 to L do
        for ℓ' = 0 to ℓ − 1 do                     ▷ Try assigning layers ℓ' + 1..ℓ to device i
            mem ← Σ_{j=ℓ'+1}^{ℓ} (p_j + k_j · S)
            if mem > m_i then
                continue                           ▷ Memory constraint violated
            end if
            t_c ← Σ_{j=ℓ'+1}^{ℓ} f_j / c_i                                  ▷ Compute time
            t_n ← a_ℓ / b_{i−1,i} + l_{i−1,i}                ▷ Network time (from prev device)
            t_stage ← max(t_c, t_n)                                         ▷ Pipeline stage time
            dp[ℓ][i] ← min(dp[ℓ][i], max(dp[ℓ'][i − 1], t_stage))
        end for
    end for
end for
π* ← Backtrack(dp)
return π*
```

Theorem 1 (Optimality)**.** *Algorithm 1 finds the partition π^* that minimizes the maximum pipeline stage latency in $O(L^2 \cdot N)$ time and $O(L \cdot N)$ space.*

Proof. The contiguity constraint ensures that any valid partition assigns a contiguous block of layers $[\ell' + 1, \ell]$ to each device i. The DP recurrence enumerates all such block boundaries. The maximum over $dp[\ell'][i-1]$ and t_{stage} correctly captures the pipeline bottleneck (the slowest stage determines throughput). Optimality follows from the optimal substructure: the best assignment of ℓ layers to i devices depends only on the best assignment of ℓ' layers to $i - 1$ devices. □

4.3 Handling Device Ordering

The DP assumes a fixed device ordering along the pipeline. We determine this ordering by solving a shortest-path problem on the device graph:

Algorithm 2 Pipeline Ordering

Require: Device graph $G = (V, E, w_v, w_e)$

Ensure: Ordered device sequence $d_1, \ldots, d_N$ minimizing total inter-stage communication

```
Construct complete graph G' with edge weight w'_ij = a/b_ij + l_ij
order ← TSP_Approximation(G')                      ▷ 2-opt local search
return order
```

For small N (typically ≤ 20), exact TSP is feasible. For larger networks, 2-opt provides near-optimal solutions.

5 Pipeline Execution Engine

5.1 Speculative Execution

To hide communication latency, we overlap activation transfer with computation on subsequent tokens:

Algorithm 3 Speculative Pipeline Execution (Per Device)

Require: Device i with layers $[\ell_{\text{start}}, \ell_{\text{end}}]$, input queue
Ensure: Processed activations forwarded to next device

```
recv_buf ← AsyncReceive()                          ▷ Non-blocking receive
loop
    h ← AwaitActivation(recv_buf)                  ▷ Wait for input
    AsyncSendPrefetch(i + 1)                       ▷ Pre-warm next device
    for ℓ = ℓ_start to ℓ_end do
        h ← LayerForward(ℓ, h)
    end for
    AsyncSend(h, device_{i+1})                     ▷ Non-blocking send
    recv_buf ← AsyncReceive()                      ▷ Pipeline: start next token
end loop
```

5.2 KV-Cache Management

The key-value cache grows with sequence length and must be managed across devices:

$$KV_size(\ell, s) = 2 \cdot d_{\text{model}} \cdot n_{\text{heads}} \cdot d_{\text{head}} \cdot s \cdot precision \tag{6}$$

Each device stores KV-cache only for its assigned layers. For a 7B model with 32 layers distributed across 4 devices (8 layers each), the KV-cache per device at sequence length 2048 is approximately 512 MB in fp16, fitting comfortably in 4+ GB RAM devices.

5.3 Quantization Strategy

We apply layer-wise quantization tailored to device capabilities:

Table 1: Quantization Configuration by Device Class

Device Class	RAM	Quantization	Bits/param	Example
Low-end mobile	2–4 GB	GPTQ 2-bit	2.5	Budget Android
Mid-range mobile	4–8 GB	GPTQ 4-bit	4.5	iPhone 13, Pixel 7
High-end mobile	8–16 GB	GPTQ 4-bit	4.5	iPhone 15 Pro
Laptop	16–32 GB	fp16	16.0	MacBook Pro
Desktop/Server	32+ GB	fp16	16.0	Gaming PC

A key insight: different pipeline stages can use different quantization levels. We assign less-quantized representations to more capable devices, placing critical layers (attention heads with highest sensitivity to quantization) on the most capable device.

Algorithm 4 Sensitivity-Aware Quantization Assignment

Require: Layer sensitivities $\{s_1, \ldots, s_L\}$, device capabilities $\{m_1, \ldots, m_N\}$
Ensure: Per-layer quantization levels $\{q_1, \ldots, q_L\}$

```
Sort layers by sensitivity: s_σ(1) ≥ s_σ(2) ≥ ...
Sort devices by capability: m_ρ(1) ≥ m_ρ(2) ≥ ...
for each device i in capability order do
    budget ← m_i
    for each layer ℓ assigned to device i do
        q_ℓ ← BestQuantization(budget, p_ℓ, s_ℓ)
        budget ← budget − p_ℓ/(32/q_ℓ)
    end for
end for
```

6 Fault Tolerance

6.1 Hot-Standby Replication

Each pipeline stage maintains a hot-standby replica on a nearby device:

Definition 4 (Standby Assignment)**.** *For each active device d_i in the pipeline, a standby device d'_i is selected such that:*

1. *d'_i has sufficient memory to hold d_i's model shard.*
2. *d'_i has low-latency connectivity to both d_{i-1} and d_{i+1}.*
3. *$d'_i \neq d_j$ and $d'_i \neq d'_j$ for all $j \neq i$ (no standby sharing).*

Algorithm 5 Failover Protocol

Require: Active device d_i becomes unresponsive
Ensure: Pipeline continues with standby d'_i

```
Detect failure: Heartbeat(d_i) = timeout after 100ms
d_{i−1} redirects output to d'_i
d'_i loads model shard from pre-cached copy
d'_i reconstructs KV-cache from last checkpoint
Resume pipeline execution
Background: Find new standby d''_i for d'_i
```

6.2 KV-Cache Checkpointing

To enable fast failover, each device periodically checkpoints its KV-cache to its standby:

Table 2: KV-Cache Checkpoint Strategy

Strategy	**Bandwidth (MB/s)**	**Failover (ms)**	**Overhead (%)**
No checkpoint	0	5,000+	0
Every token	50–200	50	15
Every 10 tokens	5–20	180	2
Every 50 tokens	1–4	800	0.4
Adaptive	2–30	180	3

The adaptive strategy checkpoints more frequently when network bandwidth is abundant and less frequently under congestion.

7 Activation Encryption

7.1 Threat Model

Intermediate activations can leak information about the input [He et al., 2019]. We protect activations in transit using lightweight encryption:

Algorithm 6 Activation Encryption

Require: Activation tensor $h \in \mathbb{R}^{s \times d}$, shared key k_{ij} between devices i, j
Ensure: Encrypted activation $\hat{h}$

```
nonce ← Counter(seq_id, layer_id)
h_bytes ← Serialize(h)                    ▷ To byte array
ĥ ← ChaCha20(k_ij, nonce, h_bytes)
return ĥ
```

ChaCha20 is chosen for its efficient software implementation on ARM devices (no AES hardware acceleration required), achieving 1+ GB/s on modern smartphones.

7.2 Key Management

Pipeline keys are established during the partitioning phase using X25519 key agreement between adjacent pipeline devices. Keys rotate every 1000 inference requests or 1 hour, whichever comes first.

8 Evaluation

8.1 Testbed Configuration

We evaluate ParsInfer on three device configurations:

Table 3: Testbed Configurations

Config	Devices	Total RAM	Network
A: Home	iPhone 15 Pro + MacBook Air M2 + iPad Pro	40 GB	Wi-Fi 6
B: Mobile	4× Pixel 8 + 4× Galaxy S23	64 GB	Wi-Fi Direct
C: Mixed	2× iPhone + Laptop + Desktop + RPi 4	58 GB	Mixed

8.2 Throughput Results

Table 4: Inference Throughput (tokens/second)

Model	Single A100	Config A	Config B	Config C	Efficiency
ParsLM-1.3B	120	95	62	78	79%
ParsLM-7B	45	33	18	27	73%
ParsLM-13B	25	16	8	14	64%

8.3 Latency Analysis

Table 5: Time-to-First-Token Latency (milliseconds)

Model	Config A	Config B	Config C	Single Device (best)
ParsLM-1.3B	180	310	240	85
ParsLM-7B	420	780	540	210
ParsLM-13B	890	1,650	1,100	450

8.4 Partitioning Quality

Table 6: Partitioning Algorithm Comparison (ParsLM-7B, Config B)

Algorithm	Throughput (tok/s)	Pipeline Imbalance	Time (ms)
Uniform (4 layers/device)	11	3.2×	0.1
Memory-proportional	14	1.8×	0.2
Compute-proportional	15	1.5×	0.3
ParsInfer DP (ours)	**18**	**1.1**×	1.2

Pipeline imbalance is defined as $\max_i T_i / \min_i T_i$ where T_i is the per-token processing time of device i.

8.5 Fault Tolerance

Table 7: Fault Tolerance Results

Failure Scenario	Recovery Time (ms)	Tokens Lost	Quality Impact
Single device departure	180	2–5	None
Two simultaneous departures	450	5–12	None
Network partition (2 groups)	2,100	20–50	None
Standby also fails	5,200	50–100	None (re-partition)

8.6 Activation Encryption Overhead

Table 8: Encryption Overhead

Method	Throughput Impact	Latency Impact	CPU Overhead
No encryption	0%	0%	0%
AES-256-GCM	−4.2%	+18ms	5.1%
ChaCha20-Poly1305	−2.8%	+12ms	3.2%
ChaCha20 (no auth)	−1.9%	+8ms	2.1%

We use ChaCha20-Poly1305 by default (authenticated encryption) with the option to disable authentication for latency-critical applications where the transport layer already provides integrity.

8.7 Comparison with Existing Systems

Table 9: Comparison with Distributed Inference Systems

Feature	ParsInfer	Petals	ExoLabs	llama.cpp	vLLM
Target devices	Mobile/Edge	GPUs	Mixed	Single device	GPUs
Heterogeneous	Yes	Partial	Yes	No	No
Fault tolerance	Hot standby	Retry	No	N/A	No
Optimal partition	DP solver	Heuristic	Manual	N/A	N/A
Activation privacy	ChaCha20	No	No	N/A	No
Mesh network	Yes	Internet	LAN	N/A	N/A
Quantization-aware	Per-stage	Global	Global	Global	No

9 Latency Optimization

9.1 Activation Compression

For bandwidth-constrained links, we optionally compress activations using lightweight learned compression:

$$\hat{h} = W_{\text{down}} \cdot h, \quad h' = W_{\text{up}} \cdot \hat{h} \tag{7}$$

where $W_{\text{down}} \in \mathbb{R}^{r \times d}$ and $W_{\text{up}} \in \mathbb{R}^{d \times r}$ with $r \ll d$. The projection matrices are trained offline to minimize reconstruction error on a calibration set.

Table 10: Activation Compression Results

Compression	Ratio	Bandwidth Saved	Quality (PPL)	Compute Cost
None	1×	0%	8.42	0
4×	4×	75%	8.48	+1%
8×	8×	87.5%	8.71	+2%
16×	16×	93.75%	9.34	+3%

9.2 Prefill-Decode Separation

The prefill phase (processing the prompt) and decode phase (generating tokens) have different computational profiles. We optimize each separately:

- **Prefill:** Compute-bound with large batch size. We maximize parallelism by distributing prompt chunks across devices.
- **Decode:** Memory-bandwidth-bound with sequential token generation. We optimize for pipeline throughput.

10 Network-Aware Scheduling

10.1 Dynamic Re-Partitioning

When network conditions change (device joins/leaves, bandwidth fluctuation), ParsInfer re-evaluates the partition:

Algorithm 7 Dynamic Re-Partitioning

Require: Current partition π, updated device graph G', running inference sessions
Ensure: Updated partition π' with minimal disruption

```
π' ← OptimalPartition(G')                          ▷ Algorithm 1
if Cost(π') < 0.8 · Cost(π) then                   ▷ Only re-partition if 20%+ improvement
    Wait for current generation to complete
    Migrate model shards: TransferLayers(π → π')
    Rebuild KV-cache on new devices
    Activate π'
end if
```

10.2 Bandwidth Probing

ParsInfer continuously probes available bandwidth between devices using lightweight ping-pong measurements:

$$\hat{b}_{ij}^{t} = (1-\alpha) \cdot \hat{b}_{ij}^{t-1} + \alpha \cdot b_{ij}^{\text{measured}} \tag{8}$$

with exponential smoothing ($\alpha = 0.1$) to avoid reacting to transient fluctuations.

11 Limitations and Future Work

1. **Training:** ParsInfer currently supports inference only. Extending to distributed fine-tuning on edge devices is future work.
2. **Tensor Parallelism:** We partition at layer boundaries only. Tensor parallelism (splitting individual layers) could improve balance for very heterogeneous configurations but requires more communication.
3. **Multimodal:** Vision and audio inputs require different partitioning strategies. We plan to extend the framework to multimodal models.
4. **Energy Efficiency:** Mobile devices have battery constraints. Energy-aware partitioning that considers power consumption is planned.
5. **Larger Models:** Models beyond 13B parameters may not fit even in aggregate memory of typical household devices. Offloading to flash storage with prefetching is under investigation.

12 Privacy and Security Analysis

12.1 Threat Model

We consider three adversary classes for distributed inference:

1. **Passive observer:** An adversary who can observe inter-device network traffic but cannot modify it. This captures ISP-level surveillance and network monitoring.
2. **Compromised node:** An adversary who controls one or more devices in the inference pipeline. They can observe activations passing through their assigned layers but cannot modify the protocol.
3. **Active attacker:** An adversary who controls devices and can inject malicious activations to influence inference results.

12.2 Activation Privacy

Theorem 2 (Activation Confidentiality)**.** *Under the ChaCha20 stream cipher [Bernstein, 2008], inter-device activations are semantically secure against passive observers. A compromised node at layer l learns only the activations at the boundary between its assigned layers, not the model input or output.*

Proof. Each inter-device activation transfer is encrypted with a per-session ChaCha20 key derived from a Diffie-Hellman key exchange between adjacent pipeline stages. ChaCha20 provides IND-CPA security under the assumption that the underlying permutation is a PRF. A compromised node at layer l observes only the decrypted activations at its input boundary (h_l) and produces outputs at its output boundary (h_{l+k}). Reconstructing the original input from intermediate activations requires inverting l transformer layers, which is computationally infeasible for $l > 1$ due to the non-invertible nature of attention mechanisms with softmax normalization. □

12.3 Inference Integrity

For applications requiring verifiable inference, ParsInfer supports an optional commitment scheme:

1. Each device commits to its computed activations before forwarding: $c_l = \mathsf{Hash}(h_l \| \text{nonce}_l)$.
2. Commitments are posted to a shared log (the Pars DHT).
3. Post-inference, any participant can request selective re-computation of specific layers to verify correctness.
4. Statistical spot-checking (5% of inference requests) provides probabilistic integrity assurance without full re-computation overhead.

13 Incentive Design

Devices contributing compute to the ParsInfer mesh earn ASHA token rewards proportional to their contribution:

$$r_i = r_{\text{base}} \cdot \frac{c_i \cdot u_i}{\sum_j c_j \cdot u_j} \tag{9}$$

where c_i is device i's compute contribution (measured in GFLOPS-hours), u_i is its uptime fraction, and r_{base} is the per-epoch reward pool. Devices that fail or produce incorrect results forfeit their reward for that epoch.

Table 11: Estimated Monthly ASHA Rewards by Device Class

Device	Contribution (GFLOPS-hr)	Uptime	Est. Reward (ASHA)
Desktop (RTX 4070)	21,024	95%	180–220
MacBook Air M2	2,592	70%	20–30
iPhone 15 Pro	1,548	40%	8–12
Pixel 8	590	45%	4–6
RPi 4	8.6	95%	0.1

14 Conclusion

ParsInfer demonstrates that distributed LLM inference on heterogeneous consumer devices is practical for interactive AI applications. By combining optimal pipeline partitioning, speculative execution, fault tolerance, and activation encryption, ParsInfer enables the Pars Network to provide community-scale AI inference without centralized infrastructure. Our evaluation shows that a collection of consumer devices can achieve 73% of single-GPU throughput with sub-second time-to-first-token latency, while maintaining bit-identical inference quality and protecting intermediate activations in transit. The incentive mechanism ensures sustainable participation by compensating device owners proportional to their contribution.

References

Borzunov, A., Baranchuk, D., Dettmers, T., et al. Petals: Collaborative inference and fine-tuning of large models. In *ACL (System Demonstrations)*, pages 38–44, 2023.

Cai, H., Zhu, L., and Han, S. ProxylessNAS: Direct neural architecture search on target task and hardware. In *ICLR*, 2019.

Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *ICLR*, 2016.

He, Z., Zhang, T., and Lee, R. B. Model inversion attacks against collaborative inference. In *ACSAC*, pages 148–162, 2019.

Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Huang, Y., Cheng, Y., Bapna, A., et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *NeurIPS*, pages 103–112, 2019.

Kang, Y., Hauswald, J., Gao, C., et al. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In *ASPLOS*, pages 615–629, 2017.

Narayanan, D., Harlap, A., Phanishayee, A., et al. PipeDream: Generalized pipeline parallelism for DNN training. In *SOSP*, pages 1–15, 2019.

Pope, R., Douglas, S., Chowdhery, A., et al. Efficiently scaling transformer inference. In *MLSys*, 2023.

Ryabinin, M., Dettmers, T., Diskin, M., and Borzunov, A. SWARM parallelism: Training large models can be surprisingly communication-efficient. In *ICML*, pages 29416–29440, 2023.

Zhao, Z., Barijough, K. M., and Gerstlauer, A. DeepThings: Distributed adaptive deep learning inference on resource-constrained IoT edge clusters. *IEEE TCAD*, 37(11):2348–2359, 2018.

Kwon, W., Li, Z., Zhuang, S., et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. GPT3.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, 2022.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.

Sheng, Y., Zheng, L., Yuan, B., et al. FlexGen: High-throughput generative inference of large language models with a single GPU. In *ICML*, pages 31094–31116, 2023.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *ICML*, pages 19274–19286, 2023.

Miao, X., Shi, O., Li, C., et al. SpotServe: Serving generative large language models on preemptible instances. In *ASPLOS*, 2024.

Shoeybi, M., Patwary, M., Puri, R., et al. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. ZeRO: Memory optimizations toward training trillion parameter models. In *SC*, pages 1–16, 2020.

Aminabadi, R. Y., Rajbhandari, S., Zhang, M., et al. DeepSpeed Inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC*, pages 1–15, 2022.

Lin, J., Tang, J., Tang, H., et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *MLSys*, 2024.

Bernstein, D. J. ChaCha, a variant of Salsa20. In *SASC Workshop*, 2008.

Ousterhout, J., Agrawal, A., Erickson, D., et al. The case for RAMCloud: Scalable high-performance storage entirely in DRAM. *ACM SIGOPS OSR*, 43(4):92–105, 2009.

ExoLabs. Exo: Run your own AI cluster at home. `https://github.com/exo-explore/exo`, 2024.

Gerganov, G. llama.cpp: Inference of LLaMA model in pure C/C++. `https://github.com/ggerganov/llama.cpp`, 2023.

A Device Benchmark Results

Table 12: Per-Device Benchmark Results (matrix multiply throughput, GFLOPS)

Device	**CPU**	**GPU/NPU**	**RAM (GB)**	**fp16 GFLOPS**	**int8 GOPS**
iPhone 15 Pro	A17 Pro	ANE 35T	8	2,150	35,000
Pixel 8	Tensor G3	Mali-G715	8	820	4,100
Galaxy S23	SD 8 Gen 2	Adreno 740	8	1,200	6,000
MacBook Air M2	M2	M2 GPU 8c	16	3,600	15,200
RPi 4	BCM2711	–	8	12	24
Desktop (RTX 4070)	i7-13700K	RTX 4070	32	29,200	233,000

B Model Partitioning Examples

For ParsLM-7B (32 layers) on Config B (8 smartphones):

Listing 1: Example Partition (ParsLM-7B on 8 devices)

```
Device 1 (Galaxy S23, 8GB): Embedding + Layers 0-3  [4 layers]
Device 2 (Galaxy S23, 8GB): Layers 4-7              [4 layers]
Device 3 (Pixel 8, 8GB):    Layers 8-11             [4 layers]
Device 4 (Pixel 8, 8GB):    Layers 12-15            [4 layers]
```

```
Device 5 (Galaxy S23, 8GB): Layers 16-19            [4 layers]
Device 6 (Galaxy S23, 8GB): Layers 20-23            [4 layers]
Device 7 (Pixel 8, 8GB):    Layers 24-27            [4 layers]
Device 8 (Pixel 8, 8GB):    Layers 28-31 + LM Head  [4 layers]

Pipeline latency per token: 55ms (18 tok/s)
Inter-device bandwidth: 120 MB/s (Wi-Fi Direct)
Activation size per transfer: 8 KB (seq_len=1, hidden=4096, fp16)
```